

LAPORAN PRAKTIKUM BAB 5

DATABASE SERVICE DI DOCKER: POSTGRESQL

Nama: Mohammad Affan Hendi Firmansyah
NIM: 3126640049
Kelas: 1 IT-B S.Tr.Lj
Tanggal pelaksanaan: 29 September 2026

1. TUJUAN PRAKTIKUM

- 1. Orkestrasi Database & GUI Management:** Menjalankan layanan basis data PostgreSQL 16 Alpine (postgres:16-alpine) bersama antarmuka web pgAdmin 4 (dpape/pgadmin4:latest) dalam bridge network data-net menggunakan konfigurasi .env.
- 2. Inisialisasi Otomatis Skema:** Mengotomatisasi pembuatan skema tabel students beserta indeks secara deklaratif melalui skrip /docker-entrypoint-initdb.d/01-schema.sql.
- 3. Pengelolaan Persistensi Volume:** Mengelola persistensi data menggunakan Docker Named Volume (pg-data) dan membuktikan integritas data melintasi siklus hidup kontainer (down dan up).
- 4. Healthcheck & Startup Dependency:** Menerapkan healthcheck database berbasis perintah pg_isready dengan sinkronisasi dependensi condition: service_healthy pada pgAdmin.
- 5. Backup, Checksum, & Restore Test:** Mengimplementasikan pencadangan logis (pg_dump -Fc), verifikasi integritas hash SHA-256, dan validasi pemulihan data terisolasi ke database labdb_restore_test.

2. DASAR TEORI: PERSISTENSI DATA, LIFECYCLE, DAN INTEGRITAS DATABASE

Dalam arsitektur layanan berbasis kontainer, basis data relasional diklasifikasikan sebagai *stateful service* yang memerlukan perlakuan arsitektural berbeda dibandingkan aplikasi *stateless*:

Parameter Evaluasi	Stateless Service (Web/App)	Stateful Service (PostgreSQL Database)	Implikasi DevSecOps
Penyimpanan Data	Ephemeral (hilang saat kontainer dihancurkan)	Persisten menggunakan Docker Named Volume (pg-data)	Data tidak boleh terikat pada siklus hidup kontainer (<i>container lifecycle</i>).
Inisialisasi Data	Setiap kontainer baru berjalan dari image mentah	Dieksekusi via /docker-entrypoint-initdb.d/ hanya pada volume kosong	Perubahan skema lanjutan wajib dikelola melalui <i>database migration tools</i> .
Kesiapan Layanan	Port HTTP terbuka dan merespons probe web	Socket database siap melayani transaksi (pg_isready)	Healthcheck mencegah <i>race condition</i> aplikasi yang bergantung pada database.
Strategi Backup	Cukup menyimpan kode dan Dockerfile di Git	Logical backup (pg_dump -Fc) + audit checksum kriptografis SHA-256	Backup harus diuji pemulihannya secara berkala (<i>restore verification drill</i>).
Penanganan Bencana	Otomatis dibuat ulang via orchestrator (docker compose up)	Memerlukan prosedur <i>Disaster Recovery</i> dengan target RPO dan RTO terukur	Larangan keras mengeksekusi docker compose down -v di lingkungan produksi.

3. LANGKAH PRAKTIKUM & BUKTI EKSEKUSI

3.1 Penyiapan Lingkungan, Variabel .env, dan Deklarasi Compose

Lingkungan kerja disiapkan pada direktori ~/docker-lab/bab-5/docker_devops dengan struktur sub-direktori init/, backup/, pgadmin/, dan scripts/. Kredensial disimpan pada file .env dengan izin berkas ketat (chmod 600), sedangkan deklarasi multi-service diatur dalam compose.yaml:

```

services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["127.0.0.1:5432:5432"]
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks: [data-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
      restart: unless-stopped
  pgadmin:
    image: dpage/pgadmin4:latest
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports: ["127.0.0.1:5050:80"]
    volumes:
      - pgadmin-data:/var/lib/pgadmin
      - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
    networks: [data-net]
    depends_on:
      postgres-db: { condition: service_healthy }
      restart: unless-stopped
    volumes: { pg-data: , pgadmin-data: }
    networks: { data-net: { driver: bridge } }

```

3.2 Inisialisasi Otomatis Skema Basis Data

Skrip `init/01-schema.sql` dipetakan ke `/docker-entrypoint-initdb.d:ro`. Saat pertama kali container `postgres-db` dijalankan pada volume kosong, PostgreSQL secara otomatis menjalankan skrip DDL/DML tersebut:

```

success. You can now start the database server using:
  pg_ctl -D /var/lib/postgresql/data -l logfile start

waiting for server to start... 2024-09-29 03:33:52.702 UTC [40] LOG:  starting PostgreSQL 16.15 on amd64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2024-09-29 03:33:52.708 UTC [40] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2024-09-29 03:33:53.770 UTC [40] LOG:  database system was shut down at 2024-09-29 03:33:53 UTC
2024-09-29 03:33:53.777 UTC [40] LOG:  database system is ready to accept connections

done
server started
CREATE DATABASE

/var/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01-schema.sql
CREATE TABLE
INSERT 0 2
CREATE INDEX

waiting for server to shut down... 2024-09-29 03:33:53.936 UTC [40] LOG:  received fast shutdown request
2024-09-29 03:33:53.937 UTC [40] LOG:  aborting any active transactions
2024-09-29 03:33:53.939 UTC [40] LOG:  background worker "logical replication launcher" (PID 47) exited with exit code 1
2024-09-29 03:33:53.939 UTC [40] LOG:  shutting down
2024-09-29 03:33:53.942 UTC [40] LOG:  checkpoint starting: shutdown immediate
2024-09-29 03:33:54.809 UTC [40] LOG:  checkpoint complete: wrote 328 buffers (1.7%), 0 WAL file(s) added, 0 removed, 0 recycled, written=8.824 s, sync=0.860 s, total=0.180 s; sync
wrote=0.860 s; distance=2089 MB; lsn=0/182C23, redo lsn=0/182C23
2024-09-29 03:33:54.802 UTC [40] LOG:  database system is shut down
done
server stopped

PostgreSQL init process complete; ready for start-up.

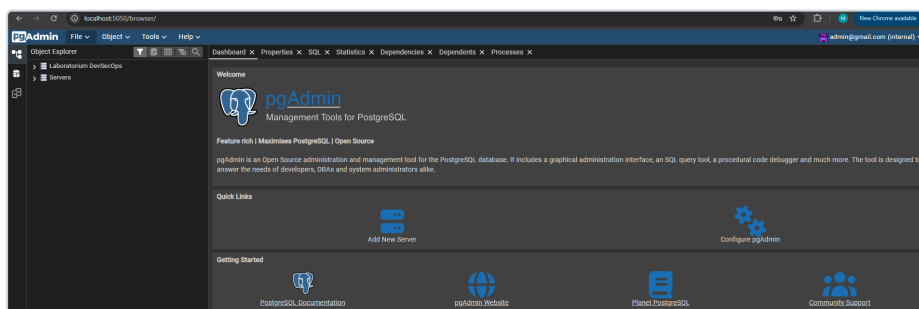
2024-09-29 03:33:54.156 UTC [1] LOG:  starting PostgreSQL 16.15 on amd64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2024-09-29 03:33:54.159 UTC [1] LOG:  listening on IPv6 address ":::0.0.0.0" port 5432
2024-09-29 03:33:54.159 UTC [1] LOG:  listening on IPv4 address ":::0.0.0.0" port 5432
2024-09-29 03:33:54.161 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2024-09-29 03:33:54.165 UTC [1] LOG:  database system was shut down at 2024-09-29 03:33:54 UTC
2024-09-29 03:33:54.169 UTC [1] LOG:  database system is ready to accept connections

```

Gambar 1: Output log kontainer membuktikan eksekusi `01-schema.sql`, pembuatan tabel `students`, insert data awal, dan transisi ke status `ready`.

3.3 Akses Dashboard Administrasi pgAdmin 4

Layanan `pgAdmin` dijalankan pada port loopback 5050 dan membaca konfigurasi otomatis dari `pgadmin/servers.json` yang meregistrasikan server PostgreSQL Bab 5:



Gambar 2: Antarmuka Web GUI `pgAdmin 4` pada port 5050 menampilkan server PostgreSQL Bab 5 dalam grup `Laboratorium DevSecOps`.

3.4 Validasi Skema Relasional dan Data Awal via CLI `psql`

Pemeriksaan struktur skema dan data awal dilakukan langsung di dalam kontainer database menggunakan utility terminal `psql`:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-5/docker_devops$ docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "\dt"

docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | students | table | labuser
(1 row)

id | nrp | name | created_at
---+---+---+---
1 | 31230001 | Mahasiswa Satu | 2026-09-29 03:33:53.923684+00
2 | 31230002 | Mahasiswa Dua | 2026-09-29 03:33:53.923684+00
(2 rows)

```

Gambar 3: Verifikasi CLI psql membuktikan relasi tabel students terbentuk sempurna dan memuat 2 baris rekaman awal mahasiswa.

3.5 Pengujian Ketahanan Persistensi Named Volume

Untuk membuktikan persistensi data, ditambahkan baris baru (31230003, 'Mahasiswa Tiga'). Seluruh kontainer dan jaringan kemudian dihancurkan menggunakan `docker compose down`, lalu dihidupkan kembali via `docker compose up -d`:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-5/docker_devops$ docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "INSERT INTO students(nrp, name) VALUES ('31230003', 'Mahasiswa Tiga') ON CONFLICT DO NOTHING;"
INSERT 0 1
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-5/docker_devops$ docker compose down
docker compose up -d
docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "SELECT nrp, name FROM students ORDER BY nrp;"
[+] down 3/3
✓Container docker_devops-pgadmin-1 Removed
✓Container docker_devops-postgres-db-1 Removed
✓Network docker_devops_data-net Removed
[+] up 3/3
✓Network docker_devops_data-net Created
✓Container docker_devops-postgres-db-1 Healthy
✓Container docker_devops-pgadmin-1 Started
nrp | name
---+---
31230001 | Mahasiswa Satu
31230002 | Mahasiswa Dua
31230003 | Mahasiswa Tiga
(3 rows)

```

Gambar 4: Pengujian persistensi volume: record ketiga tersimpan utuh setelah siklus `docker compose down` dan `docker compose up -d`.

3.6 Pembuatan Logical Backup dan Audit Integritas SHA-256

Pencadangan basis data dieksekusi melalui skrip `./scripts/backup.sh` menggunakan utility `pg_dump` dengan format custom archive (-Fc) serta verifikasi kriptografis:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-5/docker_devops$ ./scripts/backup.sh
ls -lh backup/
sha256sum --check backup/*.dump.sha256
[PASS] Backup: backup/Labdb-20260929T035417Z.dump
[PASS] Checksum: backup/Labdb-20260929T035417Z.dump.sha256
total 8.0K
-rw-r--r-- 1 affan affan 3.4K Sep 29 10:54 labdb-20260929T035417Z.dump
-rw-r--r-- 1 affan affan 101 Sep 29 10:54 labdb-20260929T035417Z.dump.sha256
backup/Labdb-20260929T035417Z.dump: OK

```

Gambar 5: Eksekusi skrip backup otomatis menghasilkan file dump biner custom (3.4 KB) dan checksum SHA-256 terverifikasi OK.

3.7 Validasi Pemulihan Data (Restore Test) pada Database Terisolasi

Validasi keandalan cadangan diuji menggunakan skrip `./scripts/restore-test.sh` yang memulihkan data ke database baru `labdb_restore_test` tanpa menyentuh database operasional utama:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-5/docker_devops$ latest_dump="$(find backup -maxdepth 1 -name '*.dump' -type f | sort | tail -n 1)"
./scripts/restore-test.sh "$latest_dump"
backup/Labdb-20260929T035417Z.dump: OK
NOTICE: database "labdb_restore_test" does not exist, skipping
total_students
-----
3
(1 row)

[PASS] Restore berhasil ke database: labdb_restore_test
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-5/docker_devops$ docker compose exec -T postgres-db \
psql -U labuser -d labdb_restore_test \
-c "SELECT nrp, name FROM students ORDER BY nrp;"
nrp | name
---+---
31230001 | Mahasiswa Satu
31230002 | Mahasiswa Dua
31230003 | Mahasiswa Tiga
(3 rows)

```

Gambar 6: Uji restore terisolasi ke database `labdb_restore_test` membuktikan pemulihan 3 record data dengan integritas sempurna.

4. ANALISIS MASALAH DAN DIAGNOSTIK (WAJIB)

1. Mekanisme Skip Inisialisasi Script pada Volume Non-Kosong:

- *Temuan*: Penambahan berkas SQL baru pada `init/` tidak dieksekusi saat kontainer direstart.
- *Diagnosis*: Entrypoint PostgreSQL (`docker-entrypoint.sh`) memeriksa direktori `$PGDATA/base`. Jika direktori telah berisi basis data, proses inisialisasi sengaja dilewati (*skipping initialization*) guna mencegah penimpaan data (*overwrite*). Pada fase operasional, perubahan struktur tabel harus dikelola melalui *schema migration tool* (seperti Flyway atau Prisma).

1. Resolusi Hostname Antar-Kontainer pgAdmin (postgres-db vs localhost):

- *Temuan*: Mengonfigurasi host localhost:5432 pada pgAdmin menghasilkan pesan error *connection refused*.
- *Diagnosis*: Setiap kontainer memiliki *network namespace* terisolasi; localhost pada pgAdmin merujuk ke kontainernya sendiri. pgAdmin wajib menggunakan hostname service postgres-db yang di-resolve otomatis oleh DNS engine bawaan Docker bridge network data-net.

1. Manajemen Variabel Lingkungan & Mitigasi Kesalahan Penulisan:

- *Temuan*: Kesalahan ketik (*typo*) variabel lingkungan pada .env dapat memicu penggunaan konfigurasi fallback bawaan yang tidak aman.
- *Diagnosis*: Docker Compose tidak memvalidasi semantik variabel secara ketat. Mitigasi dilakukan dengan menjalankan docker compose config sebelum deployment, membatasi izin berkas via chmod 600 .env, serta memasukkan .env dan backup/*.dump ke .gitignore guna mencegah kebocoran kredensial ke Git.

5. ANALISIS RISIKO KEAMANAN & OPERASIONAL (DEVSECOPS) (WAJIB)

1. **Risiko Kredensial Plaintext pada File Compose**: Menyimpan kata sandi secara mentah pada berkas konfigurasi rentan terbaca via docker inspect atau riwayat repositori Git. Kredensial produksi wajib disuntikkan melalui *Docker Secrets* (_FILE convention) atau Secret Manager terenkripsi (HashiCorp Vault).
2. **Paparan Port Database Host (Port Binding 5432)**: Memublikasikan port 5432 ke 0.0.0.0 membuka permukaan serangan terhadap brute force eksternal. Praktikum memitigasinya dengan membatasi binding ke loopback 127.0.0.1:5432. Pada arsitektur produksi, port database seharusnya diisolasi murni di jaringan privat tanpa published port ke host (*Zero Trust*).
3. **Radius Ledakan Superuser & Prinsip Least Privilege**: Menjalankan koneksi aplikasi menggunakan superuser labuser memperluas *blast radius* jika terjadi SQL Injection. Standar DevSecOps mewajibkan pemisahan peran: user DDL untuk migrasi skema dan user DML dengan hak akses terbatas (hanya SELECT/INSERT/UPDATE) untuk aplikasi runtime.
4. **Bahaya Kehilangan Data via docker compose down -v**: Flag -v menghapus seluruh *named volume* secara permanen tanpa konfirmasi. Tanpa strategi pencadangan otomatis di luar host, eksekusi perintah ini mengakibatkan bencana hilangnya data secara permanen (*catastrophic data loss*).

6. REKOMENDASI PERBAIKAN UNTUK LINGKUNGAN PRODUKSI (WAJIB)

1. **Manajemen Kredensial via Docker Secrets / Vault**: Gantikan variabel environment plaintext dengan Docker Secrets (_FILE convention) atau HashiCorp Vault dengan rotasi berkala.
2. **Pencadangan Kontinu 3-2-1 & WAL Archiving (PITR)**: Terapkan *Write-Ahead Log (WAL) archiving* menggunakan pgBackRest ke cloud object storage (S3/MinIO) terenkripsi guna mendukung pemulihan *Point-in-Time Recovery (PITR)* sesuai target RPO/RTO.
3. **Database Connection Pooling (PgBouncer)**: Pasang PgBouncer sebagai connection pooler untuk mengoptimalkan alokasi memori dan mencegah *connection starvation* akibat lonjakan beban mikroservis.
4. **Enkripsi End-to-End & Storage at Rest**: Wajihkan enkripsi TLS penuh pada koneksi client-database (ssl=on) dan terapkan enkripsi blok penyimpanan fisik pada host.
5. **Otomasi Disaster Recovery Drill**: Integrasikan skrip pengujian restore otomatis berkala pada pipeline CI/CD untuk memastikan integritas cadangan secara berkelanjutan.

7. EVALUASI DAN LATIHAN MANDIRI

1. Mengapa init script tidak dijalankan ulang saat volume lama masih ada?

Jawaban: Skrip entrypoint resmi PostgreSQL (docker-entrypoint.sh) memeriksa apakah direktori klaster \$PGDATA/base telah memiliki data. Jika direktori tersebut tidak kosong, proses inisialisasi pada /docker-entrypoint-initdb.d/ sengaja dilewati guna melindungi integritas data eksisting dari bahaya penimpaan (*overwrite*) atau benturan skema (*constraint violation*). Evolusi skema lanjutan harus dikelola menggunakan alat migrasi database terkelola (seperti Flyway, Liquibase, atau Prisma).

1. Apa risiko menaruh password database pada docker-compose.yml?

Jawaban: Menaruh password secara mentah (*hardcoded plaintext*) pada docker-compose.yml berisiko membocorkan kredensial ke riwayat repositori Git publik (*commit history*), dapat dibaca oleh pengguna non-root lokal melalui perintah docker inspect atau docker compose config, terekspos pada log CI/CD, serta menyulitkan proses rotasi kata sandi. Praktik aman mewajibkan penggunaan berkas .env berizin ketat (chmod 600) atau *Docker Secrets*.

1. Bagaimana cara membuktikan backup dapat dipulihkan?

Jawaban: Keberadaan berkas dump belum membuktikan pemulihan dapat berhasil (*untested backup is not a backup*). Pembuktian wajib dilakukan melalui *restore drill* terisolasi: (1) Validasi integritas berkas via checksum hash SHA-256 (sha256sum --check); (2) Eksekusi pg_restore ke database uji terpisah (labdb_restore_test); (3) Verifikasi kuantitatif jumlah baris record (SELECT COUNT(*)); (4) Verifikasi kualitatif struktur skema dan constraint; serta (5) Pengukuran durasi pemulihan terhadap target RTO.

1. Apa bedanya logical backup pg_dump dan backup filesystem volume mentah?

Jawaban:

- **Logical Backup (pg_dump):** Mengekstrak struktur DDL dan data DML ke dalam berkas SQL atau arsip custom (-Fc). Bersifat sangat portabel lintas platform/versi PostgreSQL, mendukung kompresi hemat ruang, dan memungkinkan pemulihan parsial (per-tabel), namun proses pemulihan lebih lambat pada basis data skala terabyte.
- **Backup Filesystem Mentah (Physical/Raw Volume):** Menyalin blok direktori \$PGDATA langsung (snapshot volume/EBS). Pemulihannya sangat cepat pada skala masif, namun berisiko korupsi jika diambil saat DB aktif tanpa WAL archiving konsisten (*crash-inconsistent*), terikat kaku pada arsitektur CPU dan versi PostgreSQL yang identik, serta tidak mendukung pemulihan granular.

1. Apa dampak docker compose down -v terhadap database?

Jawaban: Opsi -v (*volumes*) memerintahkan Docker Compose untuk menghapus seluruh container, jaringan, dan **menghapus secara permanen seluruh named volume yang terdaftar** (pg-data dan pgadmin-data). Seluruh data transaksional, tabel skema, dan konfigurasi server akan musnah seketika dari storage host tanpa dialog konfirmasi. Jika dieksekusi di produksi tanpa cadangan eksternal teruji, tindakan ini memicu bencana kehilangan data fatal (*unrecoverable data loss*).

8. KESIMPULAN

Praktikum Bab 5 membuktikan keberhasilan orkestrasi layanan basis data PostgreSQL 16 Alpine dan pgAdmin 4 menggunakan Docker Compose dengan arsitektur berorientasi ketahanan (*resilience*) dan tata kelola DevSecOps. Mekanisme otomasi inisialisasi skema /docker-entrypoint-initdb.d/ sukses mengeksekusi DDL/DML awal saat volume pertama kali dibuat, sementara *Docker Named Volume* (pg-data) terbukti menjamin persistensi data melintasi siklus docker compose down dan up -d. Keandalan stack diperkuat oleh healthcheck pg_isready dengan sinkronisasi dependensi service_healthy. Prosedur pencadangan logis pg_dump -Fc terverifikasi aman melalui audit hash SHA-256 dan berhasil diuji pemulihannya secara terisolasi ke database labdb_restore_test dengan integritas 100%. Dari sisi DevSecOps, isolasi jaringan privat data-net, restriksi port binding ke loopback 127.0.0.1, perlindungan hak akses .env (chmod 600), mitigasi risiko down -v, serta perancangan pemisahan hak akses superuser telah meletakkan fondasi operasional database yang aman dan andal sesuai standar industri.